Лабораторная работа №12. Развёртывание веб-стека на Linux-сервере

Тема: Docker Compose, Nginx Reverse Proxy, SSH, firewall, healthchecks, обновление, резервное копирование и восстановление PostgreSQL

Цель: Развернуть контейнеризированный веб-стек на отдельном Linux-сервере, проверить сетевой доступ и готовность сервисов, выполнить управляемое обновление и восстановить данные PostgreSQL из резервной копии.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №12. Итоговый проект: развёртывание веб-стека на Linux-сервере. Время: 4 академических часа (180 минут). Основной сценарий: учебный сервер Debian 13 в локальной сети с доступом по SSH и HTTP. Публичный HTTPS — дополнительный или демонстрационный этап при наличии домена.

Цель: разместить проект Nginx → Express API → PostgreSQL на отдельном Linux-хосте, ограничить внешние порты, проверить готовность сервисов, выполнить обновление API и продемонстрировать резервное копирование с восстановлением.

2. Результаты и критерии успеха

- По SSH доступен удалённый сервер; входящий порт SSH не заблокирован.
- Запущены три сервиса Compose: nginx, api, db; с клиентского ПК GET /api/products возвращает JSON.
- Снаружи опубликован только Nginx (на учебном порту 8080); API и БД доступны из внутренней сети.
- PostgreSQL имеет healthcheck; контейнер api дожидается service_healthy.
- Создан дамп базы и успешно восстановлен в отдельную тестовую БД.
- После управляемого обновления API записи в основной БД сохраняются.

3. Требуемая среда

- Подготовленная VM Debian 13 Server или VPS, доступ по SSH, sudo, доступ к Интернету для получения образов.
- Docker Engine и Docker Compose V2 из предыдущих занятий либо установленный преподавателем сервер.
- Рабочая копия проекта из лабораторной №11 с файлами compose.yaml, api/, nginx/, db/ и рабочим GET /api/products.
- Независимая копия проекта для финальной работы: compose-course12; не переиспользуйте каталог активной лабораторной.
- IP сервера заменяйте вместо SERVER_IP; имя БД и пользователя сверяйте с собственным .env/compose.yaml.
- При отсутствии публичного домена HTTPS не является обязательным для основной части.

4. Схема и исходные параметры

Сервисы и доступ

Компонент	Адрес	Доступ

Компонент	Адрес	Доступ
Клиентский ПК	http://SERVER_IP:8080/api/products	LAN/SSH-клиент
Nginx	8080 на хосте → 80 в контейнере	внешний HTTP
Express API	api:3000	только сеть Compose
PostgreSQL 17	db:5432	только сеть Compose
Docker Volume	pgdata	постоянное хранение

Модель данных — таблица products из лабораторной №8. Пример базы shop и пользователя shopuser; при другой конфигурации замените значения в командах. Не переносите .env в открытый интернет и не включайте реальные пароли в отчёт.

5. Подготовка удалённого сервера

5.1. Получение IP и проверка сервера

Подключение по SSH

```
ssh student@SERVER_IP
cat /etc/os-release
ip -br addr
sudo systemctl is-active docker
sudo docker compose version
```

Замените student и SERVER_IP своими параметрами. Проверьте, что командная строка открыта на удалённой машине, Docker active, Compose доступен. Если Docker не установлен, выполните официальную процедуру из ЛР №2, не смешивая дистрибутивные пакеты и пакеты Docker без проверки.

5.2. Правила SSH и firewall

Перед включением или изменением firewall убедитесь, что знаете фактический порт SSH и имеете резервный доступ к консоли виртуальной машины. На Debian ufw может отсутствовать: при необходимости sudo apt install ufw. На удалённом VPS проверьте также сетевые правила провайдера.

Проверка портов и настройка UFW

```
sudo ss -tlnp
sudo ufw status verbose
sudo ufw allow 22/tcp
sudo ufw allow 8080/tcp
sudo ufw status numbered
```

Если sshd использует иной порт, вместо 22/tcp разрешите его до включения UFW. Команда ufw enable допустима только после подтверждения правил и возможности восстановления доступа. Проверка: SSH-сессия продолжает работать, порт 8080 разрешён.

5.3. Передача файлов проекта

Копирование проекта на сервер (с рабочего ПК)

```
scp -r compose-course12 student@SERVER_IP:~/
```

Выполняйте scp на рабочем ПК, а не внутри сервера. Исключите локальные node_modules, .git и реальный .env с секретами других окружений. Если файлы уже на сервере, копирование не требуется.

Проверка структуры на сервере

```
cd ~/compose-course12
ls -la
sudo docker compose config -q
```

Ожидаемый результат — отсутствие сообщения об ошибке YAML. Не публикуйте в отчёте полный вывод docker compose config, если он содержит раскрытые пароли.

6. Настройка production-подобного Compose

6.1. Публикация только Nginx

В рабочем compose.yaml укажите публикацию 8080:80 только для nginx. У api и db удалите ports, если они присутствуют; для обмена между контейнерами ports не требуется. Оставьте db с именованным томом pgdata.

Пример ключевых полей Compose (ориентир, не заменяет полный проект)

```
services:
  nginx:
    ports:
      - "8080:80"
    restart: unless-stopped
  api:
    restart: unless-stopped
  db:
    image: postgres:17
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: unless-stopped
volumes:
  pgdata:
```

Фрагмент выше не является самостоятельным compose.yaml: необходимо сохранить существующие build, image, environment и network-настройки из ЛР №11. Путь /var/lib/postgresql/data относится к выбранному postgres:17.

6.2. Проверка Nginx reverse proxy

Файл nginx/default.conf (минимальный учебный вариант)

```
server {
    listen 80;
    server_name _;
    location /api/ {
        proxy_pass http://api:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
    }
}
```

Проверьте, что файл подключён в контейнер Nginx через volumes в compose.yaml и совпадает с портом Express API. В указанной форме proxy_pass сохраняет префикс /api/ в передаваемом URL.

6.3. Healthcheck PostgreSQL

Добавить к сервису db

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 20s
```

Важное замечание: $$ используется в Compose для передачи ${POSTGRES_USER} и ${POSTGRES_DB} внутрь контейнера без подстановки Compose. Healthcheck подтверждает готовность сервера PostgreSQL принимать подключения, но не гарантирует правильность таблиц или бизнес-логики.

Добавить к сервису api

```
depends_on:
  db:
    condition: service_healthy
```

Установите эти поля на правильном уровне отступов внутри api. Проверьте командой sudo docker compose config -q.

7. Запуск и проверка приложения

Сборка и запуск

```
sudo docker compose config -q
sudo docker compose up -d --build
sudo docker compose ps
```

Ожидается running для всех трёх сервисов, у db статус healthy после времени инициализации. Если какой-либо сервис упал, переходите к docker compose logs; не удаляйте том для исправления проблемы.

Проверка Nginx и API на сервере

```
sudo docker compose exec nginx nginx -t
curl -i http://127.0.0.1:8080/api/products
sudo docker compose logs --tail=30 nginx api db
```

Ожидается HTTP 200 и JSON-массив товаров. Для реальной проверки извне откройте http://SERVER_IP:8080/api/products на клиентском ПК. Если доступно только локально — проверьте firewall, сетевой адаптер VM и IP сервера.

```
Проверка отсутствия прямой публикации портов
```

```
sudo docker compose ps
sudo ss -tln | grep -E ":(8080|3000|5432)"
```

Изучите раздел PORTS docker compose ps. Наличие 3000/tcp или 5432/tcp как внутренней информации не означает публикацию на хосте; опубликованных 0.0.0.0:3000->... и 0.0.0.0:5432->... быть не должно.

8. Управляемое обновление API

Измените безопасный текст ответа в существующем GET-маршруте или добавьте диагностический route, не меняя схему БД. Перед обновлением сохраните рабочий Dockerfile, compose.yaml и предыдущую версию образа. Обновление через Compose может вызвать краткий простой.

```
Пересборка только API
```

```
sudo docker compose build api
sudo docker compose up -d --no-deps api
sudo docker compose ps
curl -f http://127.0.0.1:8080/api/products
```

Убедитесь, что nginx и db не удалялись и данные остались. Если проверка не пройдена — изучите логи api и восстановите предыдущую версию кода/образа; откат контейнера не отменяет SQL-миграции.

9. Резервное копирование и восстановление PostgreSQL

9.1. Подготовка тестовых данных

```
Записать текущее количество товаров
```

```
sudo docker compose exec -T db psql -U shopuser -d shop -c "SELECT COUNT(*) FROM
products;"
```

Запишите количество строк и несколько имён товаров. Оно потребуется для последующей проверки восстановления. Базу shop и пользователя shopuser при необходимости замените на свои.

9.2. Создание резервной копии на сервере

```
pg_dump в формате custom
```

```
mkdir -p backups
chmod 700 backups
sudo docker compose exec -T db pg_dump -U shopuser -d shop -Fc > backups/shop.dump
ls -lh backups/shop.dump
```

Используется -T (без псевдотерминала) и перенаправление >, благодаря чему файл создаётся на хосте сервера. При неудачном pg_dump файл может остаться пустым или неполным — проверьте код возврата и содержимое.

```
Проверка архива
```

```
sudo docker compose exec -T db pg_restore -l < backups/shop.dump | head -25
```

Ожидается перечень объектов БД. Архив нужно дополнительно восстановить: только просмотр оглавления не доказывает его пригодность.

9.3. Восстановление без изменения рабочей базы

```
Создание отдельной базы и восстановление

sudo docker compose exec -T db createdb -U shopuser shop_restore
sudo docker compose exec -T db pg_restore -U shopuser -d shop_restore < backups/shop.dump
sudo docker compose exec -T db psql -U shopuser -d shop_restore -c "SELECT COUNT(*) FROM
products;"
```

Если shop_restore уже существует — согласуйте с преподавателем новое уникальное имя тестовой БД; не удаляйте рабочую shop. Сравните количество и содержимое записей с результатами шага 9.1.

Фиксация результатов backup/restore

Показатель	Фактическое значение
Товаров в shop до backup	Студент заполняет
Размер файла shop.dump	Студент заполняет
Результат pg_restore	Студент заполняет
Товаров в shop_restore	Студент заполняет
Совпадают ли данные	Да/нет, доказательство

Дамп храните в защищённом каталоге с ограниченными правами. Для реального сервера нужен дополнительный отдельный носитель или удалённое защищённое хранилище: локальная копия на том же сервере не защищает от потери хоста.

10. Неисправность и восстановление

Эксперимент: остановите только API, не трогая том БД. Проверьте симптомы через HTTP и логи Nginx, затем восстановите сервис и повторите GET. Повторно не выполняйте docker compose down -v.

```
Диагностический эксперимент

sudo docker compose stop api
curl -i http://127.0.0.1:8080/api/products
sudo docker compose logs --tail=30 nginx api
sudo docker compose start api
curl -f http://127.0.0.1:8080/api/products
```

При остановленном API reverse proxy обычно возвращает 502 Bad Gateway (конкретный код зависит от конфигурации). После восстановления сервис должен снова отвечать успешно.

11. HTTPS: дополнительная часть

Если у преподавателя есть управляемый публичный домен и подготовленный сервер, продемонстрировать TLS-сертификат Let’s Encrypt с автоматическим продлением. Для HTTP-01 обычно требуется доступный из Интернета 80/tcp; проверка сертификата для localhost не является эквивалентной. В обязательной учебной части HTTP на LAN-порту 8080 достаточен.

- Проверить DNS-запись домена, входящие порты 80/443 и конфигурацию Nginx.
- Показать выпуск/монтаж сертификата и закрытый доступ к приватному ключу.
- Проверить curl -I https://DOMAIN и срок сертификата.
- Объяснить, кто перезапускает/reload Nginx после обновления сертификата.

12. Итоговая сдача и отчёт

- Схема размещения сервисов и перечень опубликованных портов.
- Скриншоты SSH, docker compose ps, health-состояния db, ответа GET /api/products с клиентского ПК.
- Фрагменты compose.yaml и nginx/default.conf без реальных секретов.
- Команды обновления API, признаки успешного запуска и логи.
- Размер дампа, результаты pg_restore в shop_restore и сравнение записей.
- Симптом, диагностика и устранение ошибки остановленного API.
- Краткий вывод: чем серверный запуск отличается от лабораторной разработки.

13. Контрольные вопросы

- Почему в публичном варианте нежелательно публиковать порты API и PostgreSQL?
- Зачем Nginx указывает имя api, а не localhost?
- Что означает service_healthy и чем он отличается от running?
- Почему docker compose up -d может сопровождаться простоем?
- Как на сервере проверить, что backup пригоден к восстановлению?
- Почему локальный Docker Volume не считается резервной копией?
- Какие риски возникают при ошибочной настройке SSH/firewall?
- Когда нужен Certbot и почему для него требуется проверка владения доменом?

14. Оценивание

Распределение баллов

Критерий	Баллы
Подготовка сервера и SSH/firewall	15
Запуск Compose, сеть и публикация через Nginx	20
Healthcheck, диагностика и HTTP-проверка	15
Обновление API и проверка	15
Backup и доказанное восстановление PostgreSQL	25
Отчёт и ответы	10
Итого	100

Требование к безопасной работе: не допускается выполнение docker compose down -v ради исправления ошибки. Оценка HTTPS не входит в обязательные 100 баллов, если в учебной среде нет публичного домена. Все фактические результаты (статусы, объём дампа, число строк) студент получает самостоятельно.